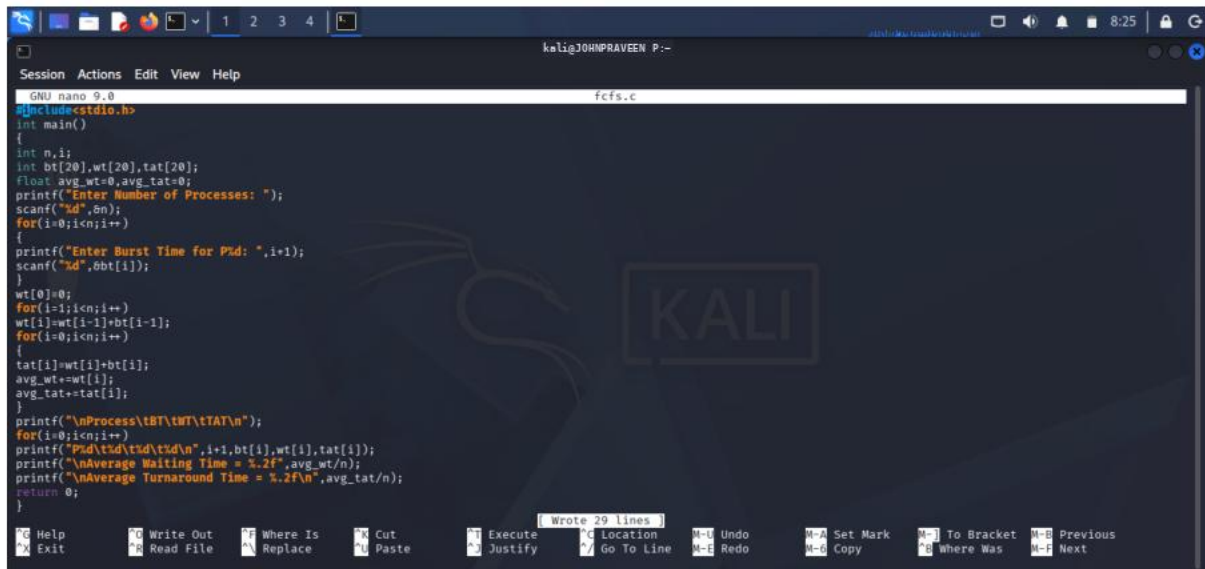


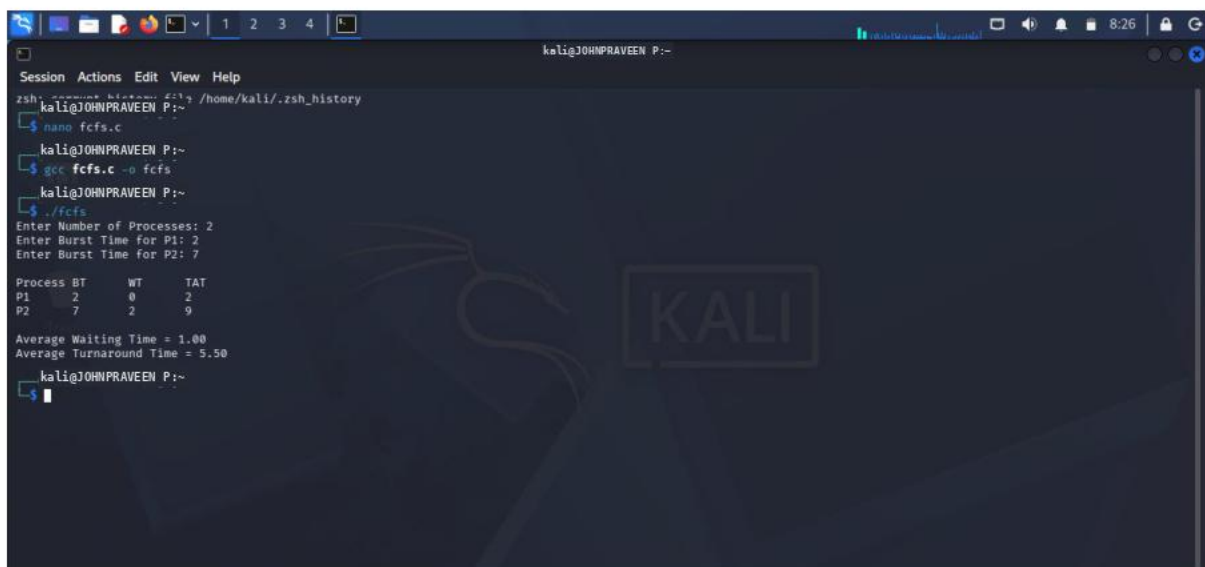
C PROGRAMMING : FCFS



The screenshot shows a terminal window with the nano text editor open. The file being edited is named 'fcfs.c'. The code is a C program that implements the First-Come, First-Served (FCFS) scheduling algorithm. It prompts the user to enter the number of processes, then the burst time for each process. It calculates the waiting time (WT) and turnaround time (TAT) for each process and displays the results in a table. The code also calculates the average waiting time and average turnaround time.

```
GNU nano 9.0 fcfs.c
#include<stdio.h>
int main()
{
    int n,i;
    int bt[20],wt[20],tat[20];
    float avg_wt=0,avg_tat=0;
    printf("Enter Number of Processes: ");
    scanf("%d",&n);
    for(i=0; i<n; i++)
    {
        printf("Enter Burst Time for P%d: ",i+1);
        scanf("%d",&bt[i]);
    }
    wt[0]=0;
    for(i=1; i<n; i++)
    {
        wt[i]=wt[i-1]+bt[i-1];
    }
    for(i=0; i<n; i++)
    {
        tat[i]=wt[i]+bt[i];
        avg_wt+=wt[i];
        avg_tat+=tat[i];
    }
    printf("\nProcess\tBT\tWT\tTAT\n");
    for(i=0; i<n; i++)
    {
        printf("P%d\t%d\t%d\t%d\n",i+1,bt[i],wt[i],tat[i]);
    }
    printf("\nAverage Waiting Time = %.2f",avg_wt/n);
    printf("\nAverage Turnaround Time = %.2f",avg_tat/n);
    return 0;
}
```

OUTPUT :



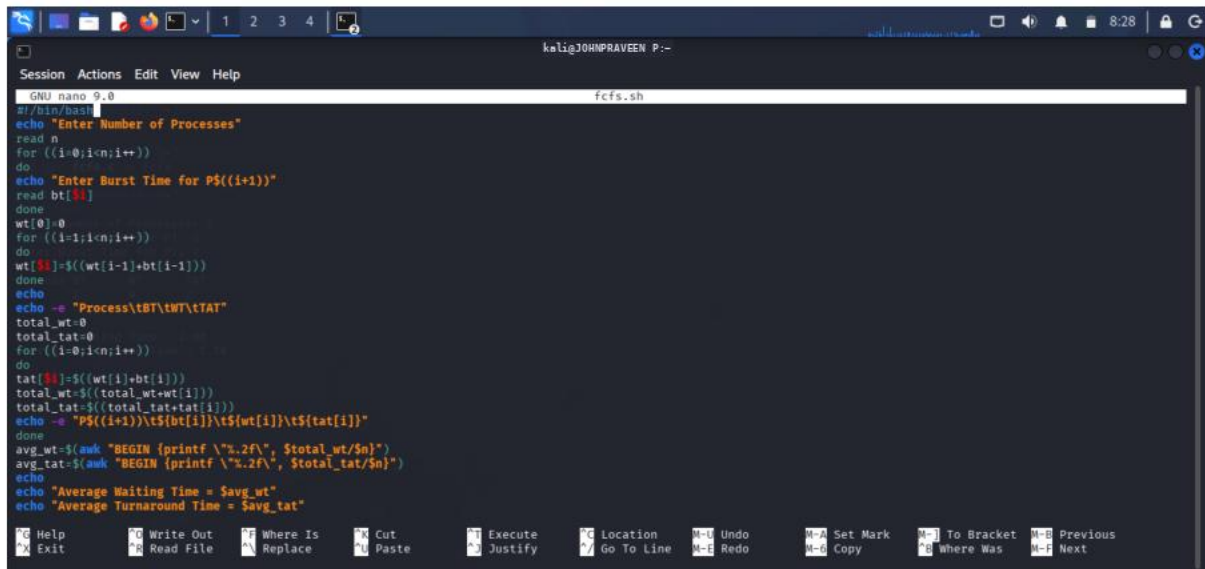
The screenshot shows a terminal window with the output of the FCFS program. The user has entered 2 processes. The first process has a burst time of 2, and the second process has a burst time of 7. The program outputs the waiting time (WT) and turnaround time (TAT) for each process, and the average waiting time and average turnaround time.

```
zsh: /home/kali/.zsh_history
kali@JOHNRAVEEN P:~$ nano fcfs.c
kali@JOHNRAVEEN P:~$ gcc fcfs.c -o fcfs
kali@JOHNRAVEEN P:~$ ./fcfs
Enter Number of Processes: 2
Enter Burst Time for P1: 2
Enter Burst Time for P2: 7

Process BT    WT    TAT
P1      2      0      2
P2      7      2      9

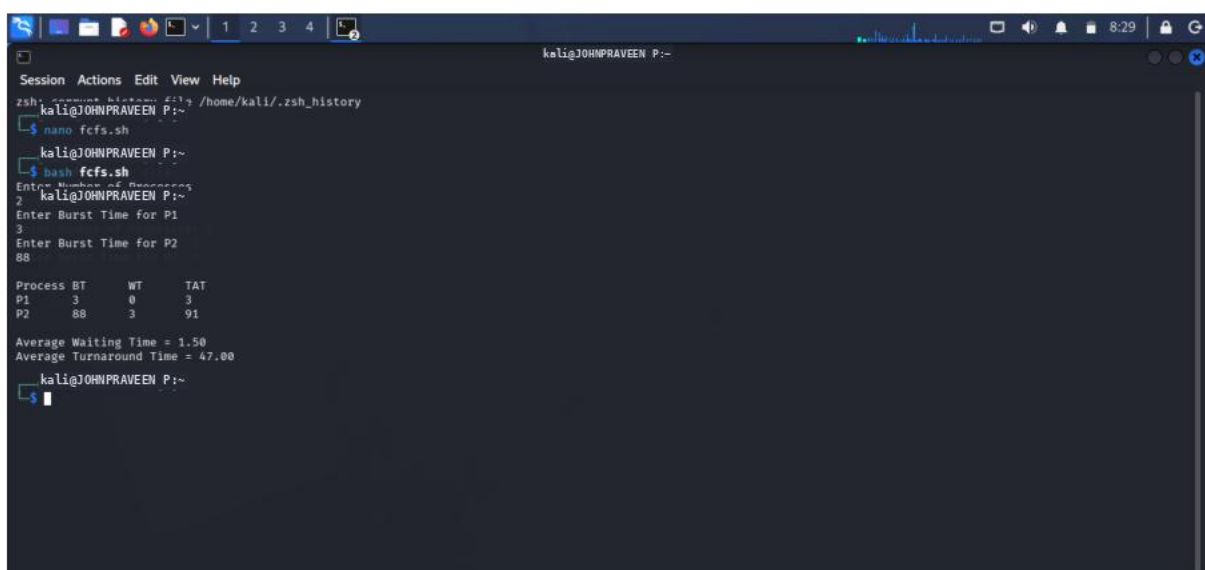
Average Waiting Time = 1.00
Average Turnaround Time = 5.50
kali@JOHNRAVEEN P:~$
```

SHELL PROGRAMMING :FCFS



```
GNU nano 9.0 fcfs.sh
#!/bin/bash
echo "Enter Number of Processes"
read n
for ((i=0;i<n;i++))
do
echo "Enter Burst Time for P$((i+1))"
read bt[i]
done
wt[0]=0
for ((i=1;i<n;i++))
do
wt[i]=${wt[i-1]}+bt[i-1]
done
echo -e "Process\tBT\tWT\tTAT"
total_wt=0
total_tat=0
for ((i=0;i<n;i++))
do
tat[i]=${wt[i]}+bt[i]
total_wt=$((total_wt+wt[i]))
total_tat=$((total_tat+tat[i]))
echo -e "P$((i+1))\t${bt[i]}\t${wt[i]}\t${tat[i]}"
done
avg_wt=$(awk "BEGIN {printf \"%.2f\", $total_wt/$n}")
avg_tat=$(awk "BEGIN {printf \"%.2f\", $total_tat/$n}")
echo
echo "Average Waiting Time = $avg_wt"
echo "Average Turnaround Time = $avg_tat"
```

OUTPUT :

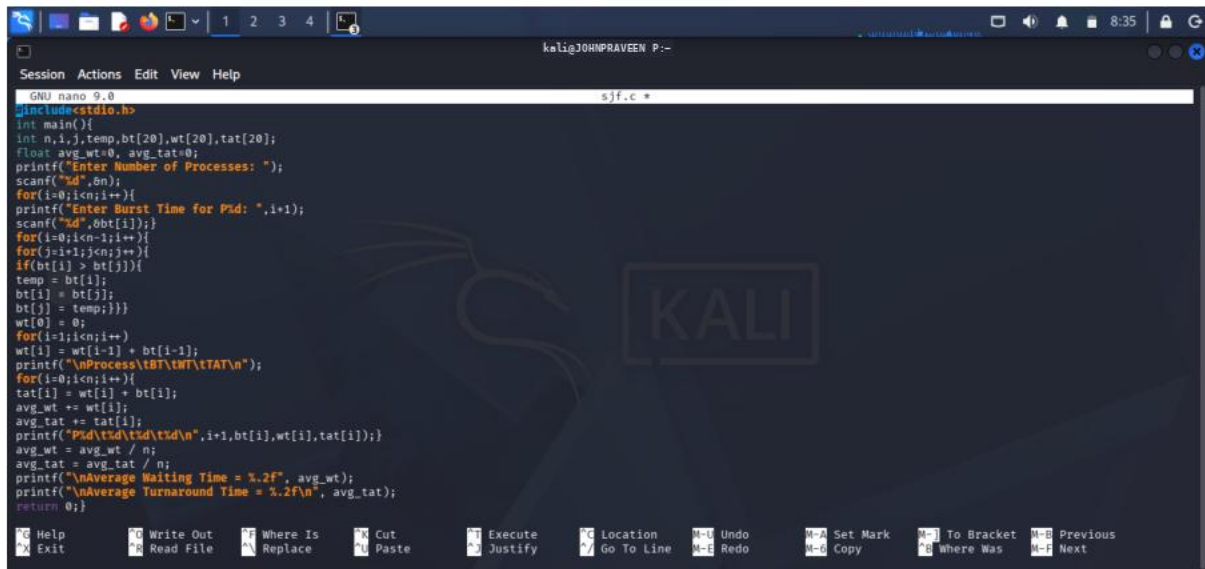


```
zsh: zsh history file: /home/kali/.zsh_history
kali@JOHNPRAVEEN P:~$ nano fcfs.sh
kali@JOHNPRAVEEN P:~$ bash fcfs.sh
Enter Number of Processes
2
Enter Burst Time for P1
3
Enter Burst Time for P2
88

Process BT    WT    TAT
P1      3      0      3
P2     88      3     91

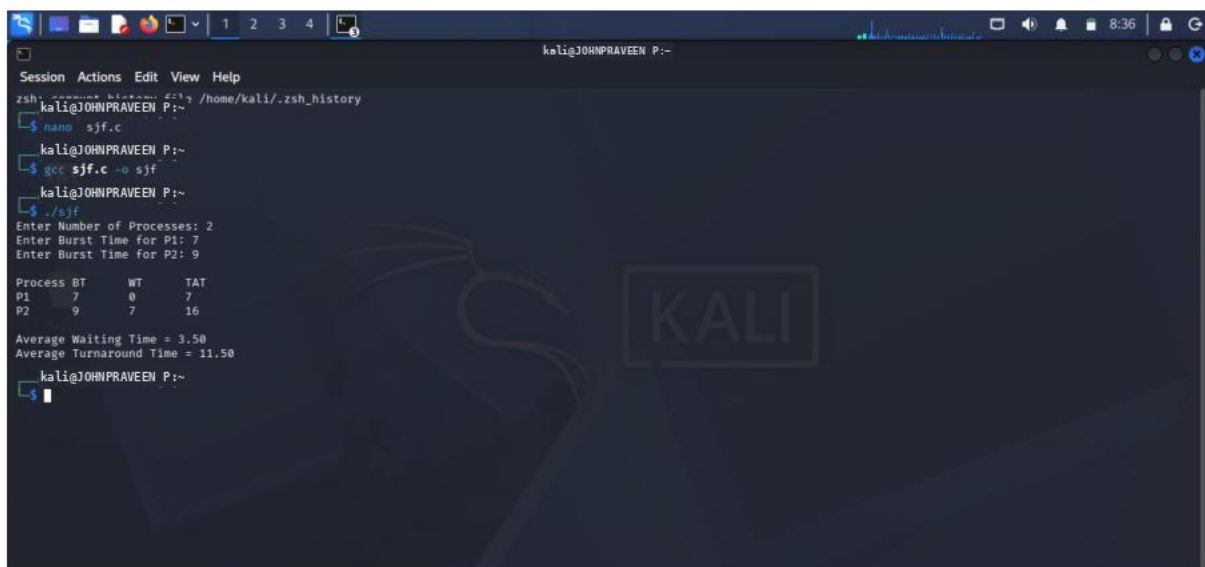
Average Waiting Time = 1.50
Average Turnaround Time = 47.00
kali@JOHNPRAVEEN P:~$
```

C PROGRAMMING :SJF



```
GNU nano 9.0 sjf.c *
#include<stdio.h>
int main(){
    int n,i,j,temp,bt[20],wt[20],tat[20];
    float avg_wt=0, avg_tat=0;
    printf("Enter Number of Processes: ");
    scanf("%d",&n);
    for(i=0;i<n;i++){
        printf("Enter Burst Time for P%d: ",i+1);
        scanf("%d",&bt[i]);
        for(j=i+1;j<n;j++){
            if(bt[i] > bt[j]){
                temp = bt[i];
                bt[i] = bt[j];
                bt[j] = temp;
            }
        }
        wt[i] = 0;
        for(i=1;i<n;i++){
            wt[i] = wt[i-1] + bt[i-1];
            printf("\nProcess %d\tBT\tWT\tTAT\n",i);
            for(i=0;i<n;i++){
                tat[i] = wt[i] + bt[i];
                avg_wt += wt[i];
                avg_tat += tat[i];
                printf("P%d\t%d\t%d\t%d\n",i+1,bt[i],wt[i],tat[i]);
            }
            avg_wt = avg_wt / n;
            avg_tat = avg_tat / n;
            printf("\nAverage Waiting Time = %.2f", avg_wt);
            printf("\nAverage Turnaround Time = %.2f\n", avg_tat);
        }
        return 0;
    }
```

OUTPUT :



```
zsh: /home/kali/.zsh_history
kali@JOHNRAVEEN P:~
$ nano sjf.c
kali@JOHNRAVEEN P:~
$ gcc sjf.c -o sjf
kali@JOHNRAVEEN P:~
$ ./sjf
Enter Number of Processes: 2
Enter Burst Time for P1: 7
Enter Burst Time for P2: 9

Process BT    WT    TAT
P1      7      0      7
P2      9      7     16

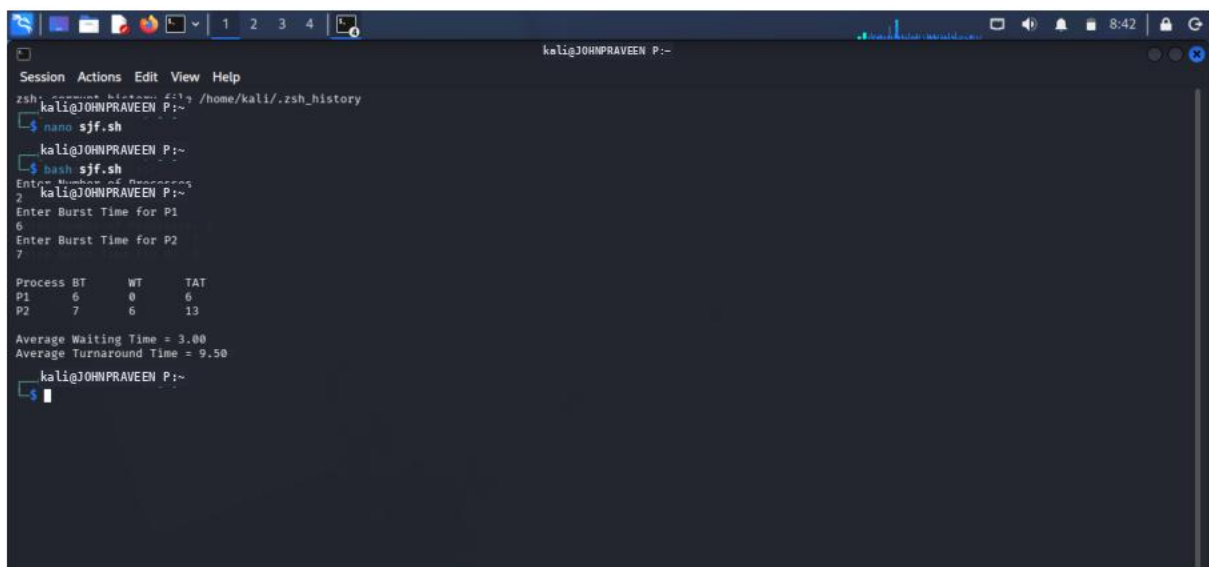
Average Waiting Time = 3.50
Average Turnaround Time = 11.50
kali@JOHNRAVEEN P:~
$
```

SHELL PROGRAMMING : SJF



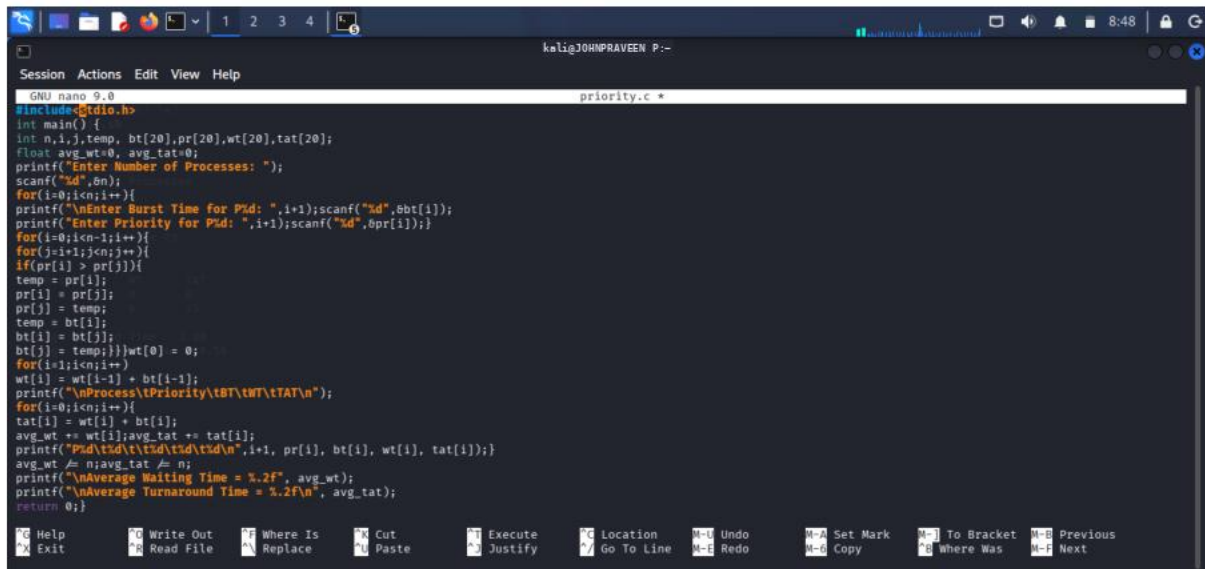
```
GNU nano 9.0 sjf.sh *
#!/bin/bash
echo "Enter Number of Processes"
read n
for ((i=0;i<n;i++))
do
echo "Enter Burst Time for P${(i+1)}"
read bt[i]
done
for ((i=0;i<n-1;i++))
do
for ((j=i+1;j<n;j++))
do
if [ ${bt[i]} -gt ${bt[j]} ]
then
temp=${bt[i]}
bt[i]=${bt[j]}
bt[j]=$temp
fi
done
done
wt[0]=0
echo
echo -e "Process\tBT\tWT\tTAT"
total_wt=0
total_tat=0
for ((i=0;i<n;i++))
do
if [ $i -ne 0 ]
then
```

OUTPUT :



```
zsh: command not found: nano /home/kali/.zsh_history
kali@JOHNRAVEEN P:~$ nano sjf.sh
kali@JOHNRAVEEN P:~$ bash sjf.sh
Enter Number of Processes
2
Enter Burst Time for P1
6
Enter Burst Time for P2
7
Process BT      WT      TAT
P1      6       0       6
P2      7       6      13
Average Waiting Time = 3.00
Average Turnaround Time = 9.50
kali@JOHNRAVEEN P:~$
```

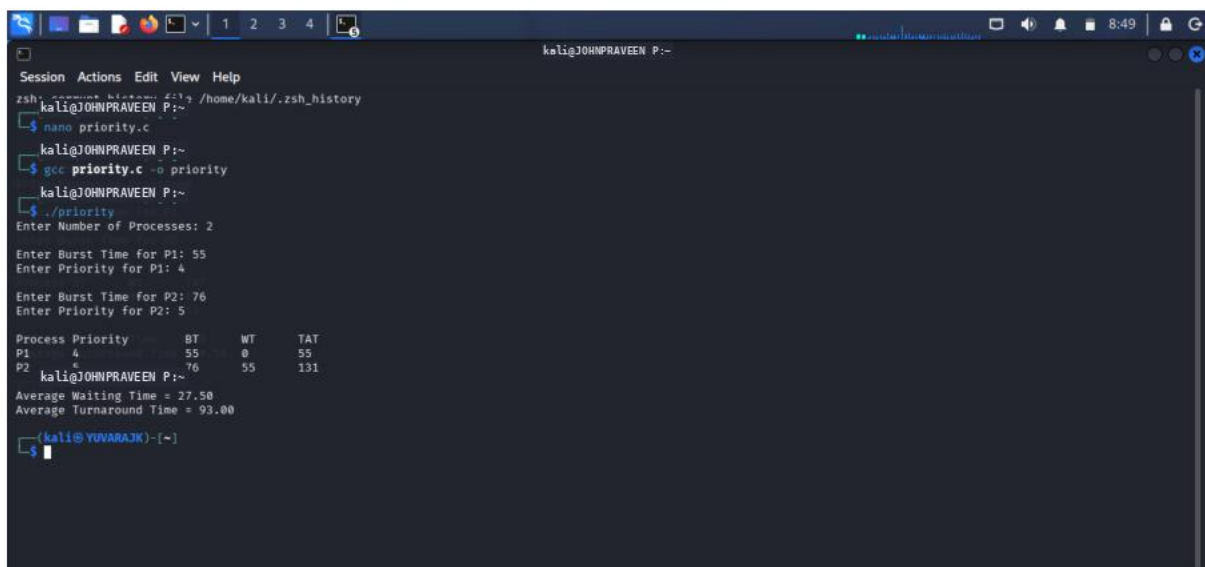
C PROGRAMMING : PRIORITY SCHEDULING



The screenshot shows a nano 9.0 editor window titled 'priority.c *' on a Kali Linux terminal. The code implements a priority scheduling algorithm. It takes the number of processes (n) as input, then for each process, it takes its burst time (bt) and priority (pr). The processes are sorted by priority. The waiting time (wt) and turnaround time (tat) are calculated for each process based on its position in the sorted order. The average waiting time and average turnaround time are also calculated and printed.

```
GNU nano 9.0 priority.c *
#include<stdio.h>
int main() {
    int n,i,j,temp, bt[20],pr[20],wt[20],tat[20];
    float avg_wt=0, avg_tat=0;
    printf("Enter Number of Processes: ");
    scanf("%d",&n);
    for(i=0;i<n;i++){
        printf("\nEnter Burst Time for P%d: ",i+1);scanf("%d",&bt[i]);
        printf("Enter Priority for P%d: ",i+1);scanf("%d",&pr[i]);
        for(j=i+1;j<n;j++){
            if(pr[i] > pr[j]){
                temp = pr[i];
                pr[i] = pr[j];
                pr[j] = temp;
                temp = bt[i];
                bt[i] = bt[j];
                bt[j] = temp;}}wt[0] = 0;
        for(i=1;i<n;i++){
            wt[i] = wt[i-1] + bt[i-1];
            printf("\nProcess\tPriority\tBT\tWT\tTAT\n");
            for(i=0;i<n;i++){
                tat[i] = wt[i] + bt[i];
                avg_wt += wt[i];avg_tat += tat[i];
            }printf("P%d\t%d\t%d\t%d\t%d\n",i+1, pr[i], bt[i], wt[i], tat[i]);
            avg_wt /= n;avg_tat /= n;
        }printf("\nAverage Waiting Time = %.2f", avg_wt);
        printf("\nAverage Turnaround Time = %.2f\n", avg_tat);
        return 0;
}
```

OUTPUT :



The screenshot shows the terminal output of the program. It starts with the user entering the number of processes as 2. Then, for each process, the burst time and priority are entered. The output shows the sorted processes with their respective burst times, waiting times, and turnaround times. Finally, the average waiting time and average turnaround time are calculated and displayed.

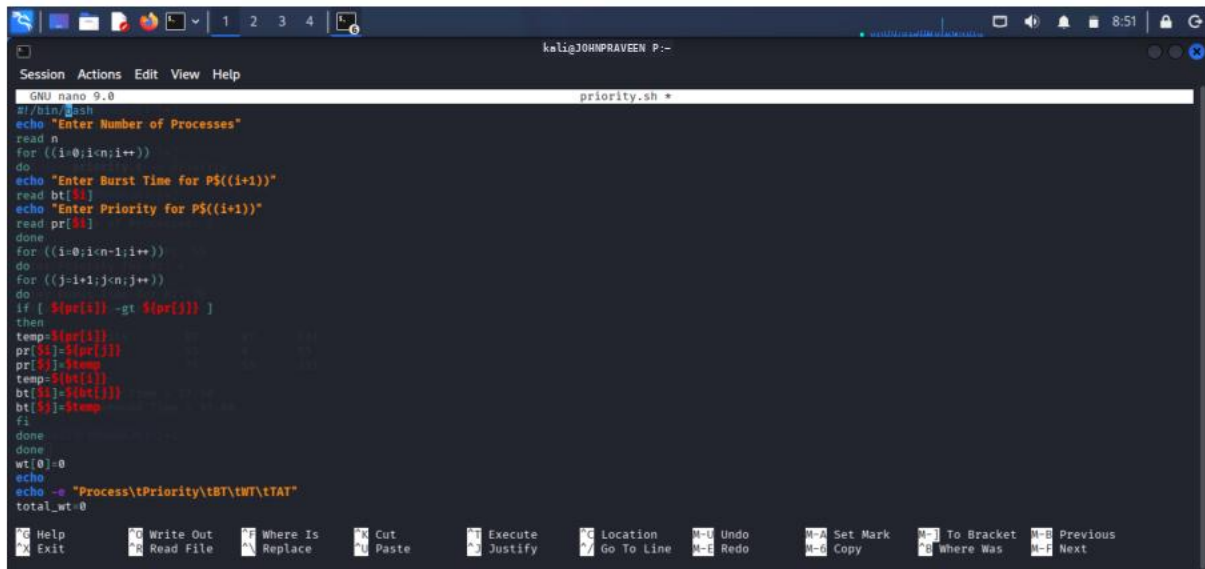
```
zsh: /home/kali/.zsh_history
kali@JOHNRAVEEN P:~
$ nano priority.c
kali@JOHNRAVEEN P:~
$ gcc priority.c -o priority
kali@JOHNRAVEEN P:~
$ ./priority
Enter Number of Processes: 2

Enter Burst Time for P1: 55
Enter Priority for P1: 4

Enter Burst Time for P2: 76
Enter Priority for P2: 5

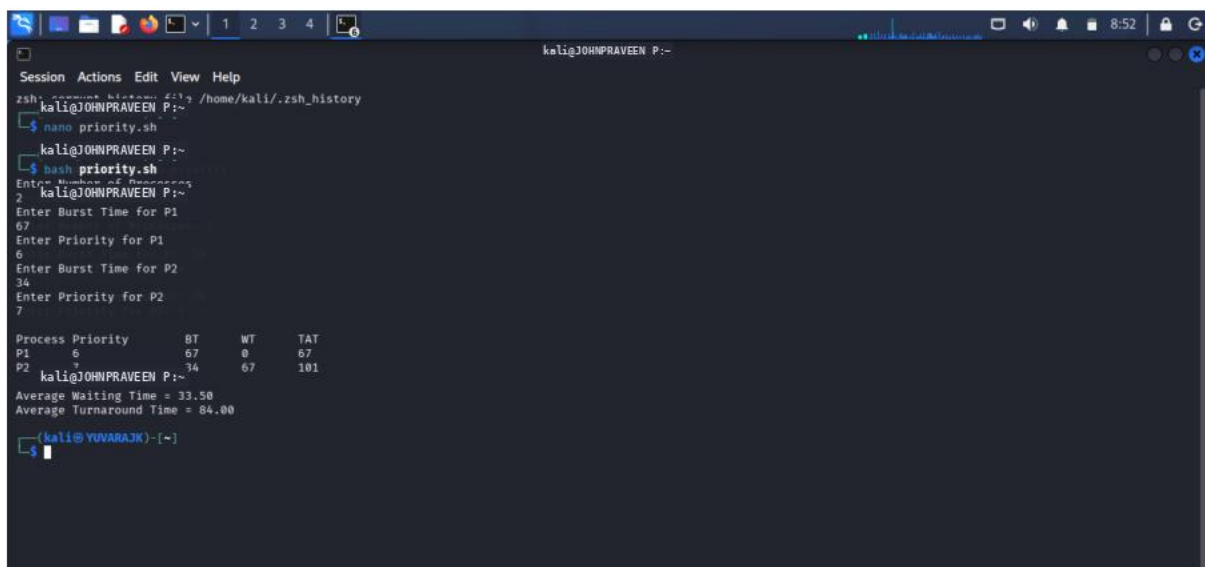
Process Priority      BT      WT      TAT
P1        4         55        0       55
P2        5         76        55      131
kali@JOHNRAVEEN P:~
Average Waiting Time = 27.50
Average Turnaround Time = 93.00
kali@YUVARAJK:~
$
```

SHELL PROGRAMMING : SJF



```
GNU nano 9.0 priority.sh
#!/bin/bash
echo "Enter Number of Processes"
read n
for ((i=0;i<n;i++))
do
echo "Enter Burst Time for P${(i+1)}"
read bt[i]
echo "Enter Priority for P${(i+1)}"
read pr[i]
done
for ((i=0;i<n-1;i++))
do
for ((j=i+1;j<n;j++))
do
if [ ${pr[i]} -gt ${pr[j]} ]
then
temp=${pr[i]}
pr[i]=${pr[j]}
pr[j]=$temp
temp=${bt[i]}
bt[i]=${bt[j]}
bt[j]=$temp
fi
done
done
wt[0]=0
echo
echo -e "Process\tPriority\tBT\tWT\tTAT"
total_wt=0
```

OUTPUT :

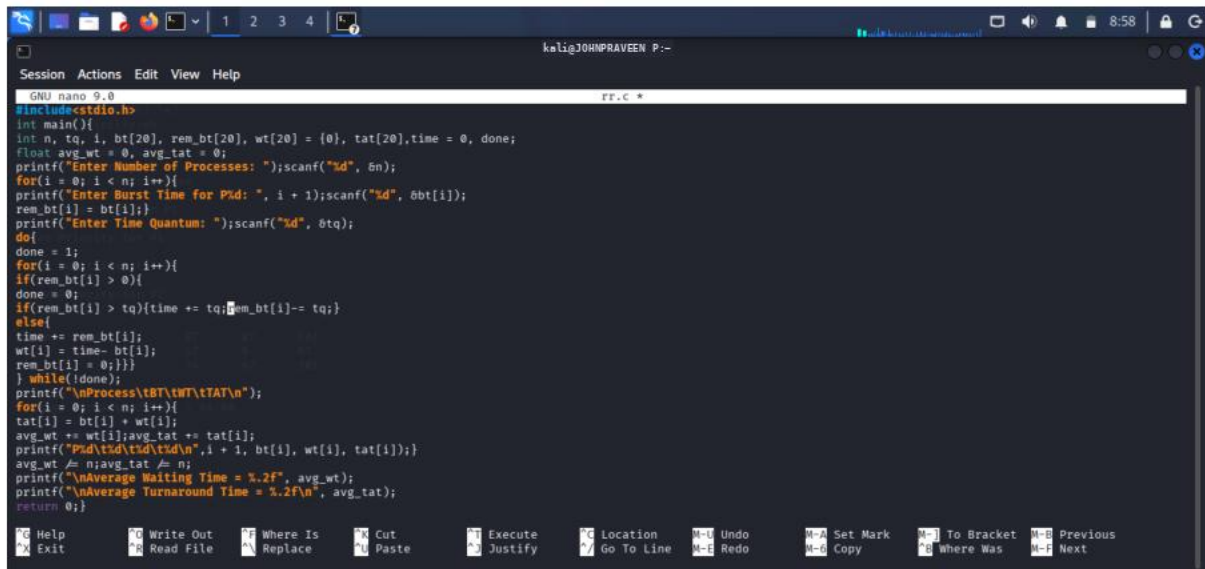


```
zsh: /home/kali/.zsh_history
kali@JOHNPRAVEEN P:~
$ nano priority.sh
kali@JOHNPRAVEEN P:~
$ bash priority.sh
Enter Number of Processes
2
Enter Burst Time for P1
67
Enter Priority for P1
6
Enter Burst Time for P2
34
Enter Priority for P2
7

Process Priority      BT      WT      TAT
P1          6        67         67
P2          7        34        101

kali@JOHNPRAVEEN P:~
Average Waiting Time = 33.50
Average Turnaround Time = 84.00
kali@YUVARAJK:~
$
```

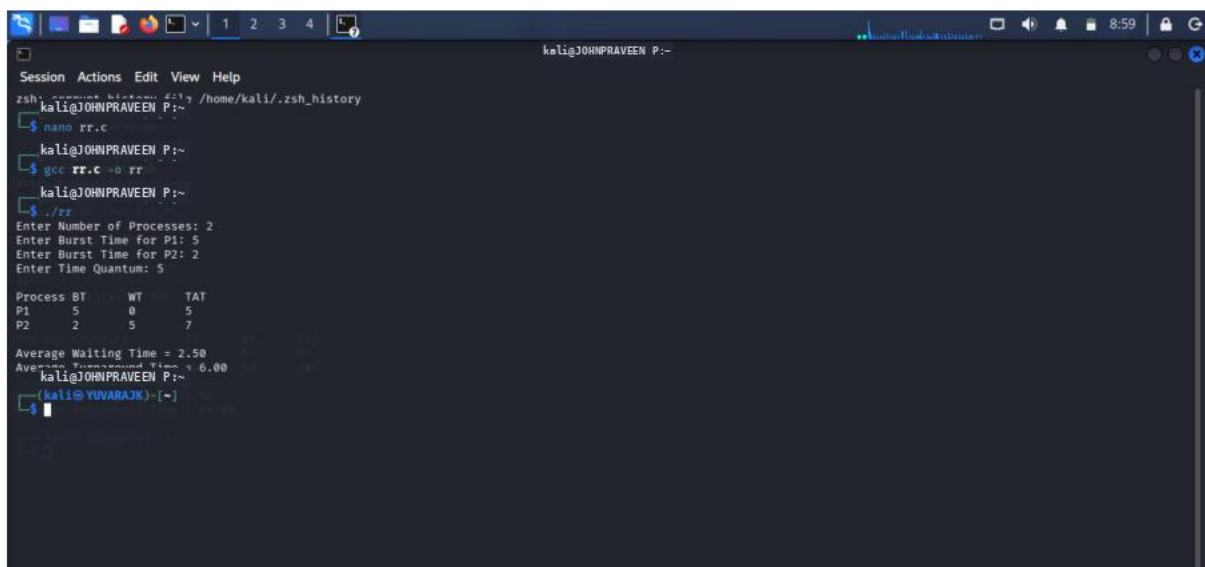
C PROGRAMMING : ROUND ROBIN



The screenshot shows a terminal window with the nano text editor open, editing a file named rr.c. The code implements a Round Robin scheduling algorithm. It takes the number of processes (n), their burst times (bt), and a time quantum (tq) as input. The program calculates the waiting time (wt) and turnaround time (tat) for each process and prints the average waiting and turnaround times.

```
GNU nano 9.0 rr.c
#include<stdio.h>
int main(){
    int n, tq, i, bt[20], rem_bt[20], wt[20], tat[20], time = 0, done;
    float avg_wt = 0, avg_tat = 0;
    printf("Enter Number of Processes: ");scanf("%d", &n);
    for(i = 0; i < n; i++){
        printf("Enter Burst Time for P%d: ", i + 1);scanf("%d", &bt[i]);
        rem_bt[i] = bt[i];
        printf("Enter Time Quantum: ");scanf("%d", &tq);
        do{
            done = 1;
            for(i = 0; i < n; i++){
                if(rem_bt[i] > 0){
                    done = 0;
                    if(rem_bt[i] > tq){time += tq; rem_bt[i] -= tq;}
                    else{
                        time += rem_bt[i];
                        wt[i] = time - bt[i];
                        rem_bt[i] = 0;
                    }
                }
            }
            printf("\nProcess\tBT\tWT\tTAT\n");
            for(i = 0; i < n; i++){
                tat[i] = bt[i] + wt[i];
                avg_wt += wt[i];avg_tat += tat[i];
                printf("P%d\t%d\t%d\t%d\n", i + 1, bt[i], wt[i], tat[i]);
            }
            avg_wt /= n;avg_tat /= n;
            printf("\nAverage Waiting Time = %.2f", avg_wt);
            printf("\nAverage Turnaround Time = %.2f\n", avg_tat);
        } while(!done);
    }
    return 0;
}
```

OUTPUT :



The screenshot shows the terminal output of the Round Robin scheduling program. The user enters 2 processes, with burst times of 5 and 2, and a time quantum of 5. The program outputs a table of process details and the average waiting and turnaround times.

```
zsh: /home/kali/.zsh_history
kali@JOHNPRAVEEN P:~$ nano rr.c
kali@JOHNPRAVEEN P:~$ gcc rr.c -o rr
kali@JOHNPRAVEEN P:~$ ./rr
Enter Number of Processes: 2
Enter Burst Time for P1: 5
Enter Burst Time for P2: 2
Enter Time Quantum: 5

Process BT    WT    TAT
P1      5      0      5
P2      2      5      7

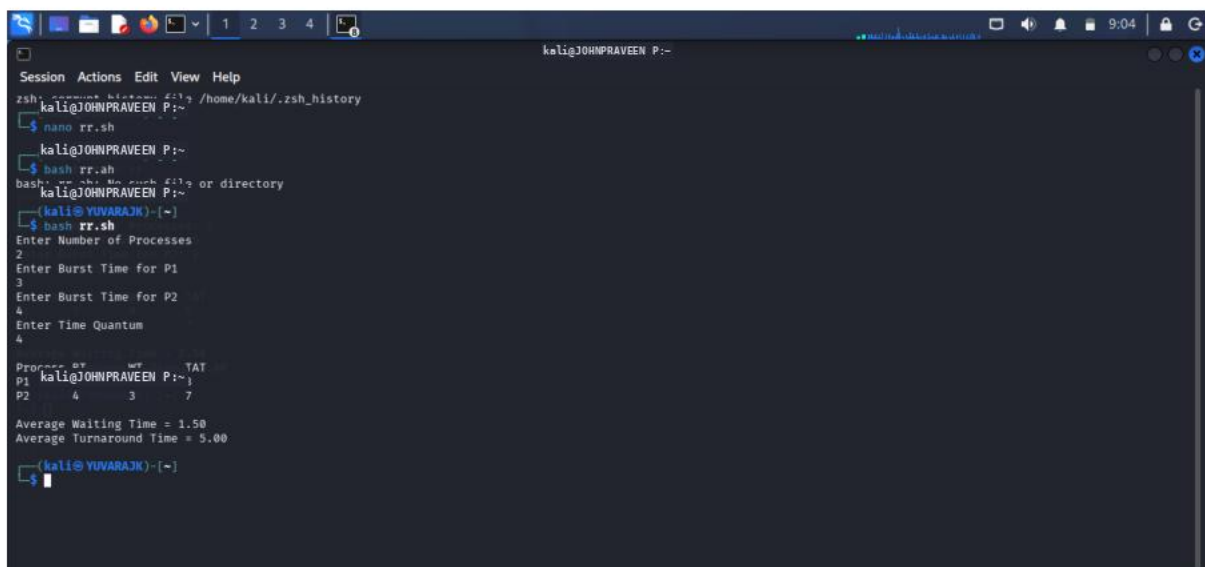
Average Waiting Time = 2.50
Average Turnaround Time = 6.00
kali@JOHNPRAVEEN P:~$
```


SHELL PROGRAMMING : ROUND ROBIN



```
GNU nano 9.0 rr.sh *
#!/bin/bash
echo "Enter Number of Processes"
read n
for ((i=0;i<n;i++))
do
echo "Enter Burst Time for P$((i+1))"
read bt[$i]
rem_bt=$((bt-$bt))
done
echo "Enter Time Quantum"
read tq
time=0
while true
do
done=1
for ((i=0;i<n;i++))
do
if [ ${rem_bt[i]} -gt 0 ]
then
done=0
if [ ${rem_bt[i]} -gt $tq ]
then
time=$((time+tq))
rem_bt[i]=$((rem_bt[i]-tq))
else
time=$((time+rem_bt[i]))
wt[$i]=$((time-bt[i]))
rem_bt[i]=0
fi
fi
done
done
```

OUTPUT :



```
zsh: command not found: /home/kali/.zsh_history
kali@JOHNPRAVEEN P:~$ nano rr.sh
kali@JOHNPRAVEEN P:~$ bash rr.sh
bash: rr.sh: No such file or directory
kali@JOHNPRAVEEN P:~$ bash rr.sh
Enter Number of Processes
2
Enter Burst Time for P1
3
Enter Burst Time for P2
4
Enter Time Quantum
4

Proc---BT---WT---TAT
P1 3---0---3---7
P2 4---3---7---11

Average Waiting Time = 1.50
Average Turnaround Time = 5.00
kali@YUVARAJK-[-]$
```